

```
from transformers import T5Tokenizer, T5ForConditionalGeneration  
pip install sentencepiece
```

Requirement already satisfied: sentencepiece in
/usr/local/lib/python3.11/dist-packages (0.2.0)

```
from transformers import T5Tokenizer, T5ForConditionalGeneration
```

```
# Load T5-small model and tokenizer
```

```
model_name = "t5-small"
```

```
tokenizer = T5Tokenizer.from_pretrained(model_name)
```

```
model = T5ForConditionalGeneration.from_pretrained(model_name)
```

```
# Move model to GPU if available
```

```
import torch
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model.to(device)
```

```
print("T5-small model loaded successfully!")
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/
```

```
_auth.py:94: UserWarning:
```

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your session.

You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
```

```
{"model_id": "f2bca518787b467f9633d2047dcb593", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "05c510148b7a4328918f5bdbef46a898", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "973ab1daf4f4410a9e65d2159e471b2a", "version_major": 2, "version_minor": 0}
```

You are using the default legacy behaviour of the `<class 'transformers.models.t5.tokenization_t5.T5Tokenizer'>`. This is expected, and simply means that the `legacy` (previous) behavior will be used so nothing changes for you. If you want to use the new behaviour, set `legacy=False`. This should only be set if you understand what it means, and thoroughly read the reason why this was added as explained in

<https://github.com/huggingface/transformers/pull/24565>

```
{"model_id": "8408d6f96c6d4cfd8415dcf1cd34d180", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f3fc4bef78264597b6cd1054aeefa6f0", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "7296f410d3a3432cb7e121fefacd7f4d", "version_major": 2, "version_minor": 0}
```

T5-small model loaded successfully!

```
def summarize_text(text, max_length=150):  
    # Preprocess the input text  
    input_ids = tokenizer(  
        "summarize: " + text,  
        return_tensors="pt",  
        max_length=512,  
        truncation=True  
    ).input_ids.to(device)  
  
    # Generate summary  
    summary_ids = model.generate(  
        input_ids,  
        max_length=max_length,  
        min_length=50,  
        length_penalty=2.0,  
        num_beams=4,  
        early_stopping=True  
    )  
  
    # Decode and return the summary  
    return tokenizer.decode(summary_ids[0], skip_special_tokens=True)
```

Example text

```
text = """The CNN/DailyMail dataset is one of the most commonly used  
datasets for abstractive text summarization.  
It contains news articles along with human-written summaries. The  
dataset is widely used to train models like BART,  
T5, and Pegasus. The main challenge in summarization is to balance  
content compression while maintaining factual correctness."""
```

Generate summary

```
summary = summarize_text(text)  
print("Generated Summary:")  
print(summary)
```

Generated Summary:

the dataset is widely used to train models like BART, T5 and Pegasus.
the main challenge in summarization is to balance content compression
while maintaining factual correctness. the dataset is widely used to
train models like BART, T5 and pegasus.

```
def summarize_text(text, max_length=100, min_length=30):  
    # Tokenize input text
```

```

input_ids = tokenizer(
    "summarize: " + text,
    return_tensors="pt",
    max_length=512,
    truncation=True
).input_ids.to(device)

# Generate summary with adjusted parameters
summary_ids = model.generate(
    input_ids,
    max_length=max_length,
    min_length=min_length,
    length_penalty=1.0, # Adjusted to avoid repetition
    num_beams=5, # Increased beams for better output
    repetition_penalty=2.5, # Prevents repeating words
    early_stopping=True
)

# Decode and return the summary
return tokenizer.decode(summary_ids[0], skip_special_tokens=True)

```

Example input text

text = """The CNN/DailyMail dataset is one of the most commonly used datasets for abstractive text summarization. It contains news articles along with human-written summaries. The dataset is widely used to train models like BART, T5, and Pegasus. The main challenge in summarization is to balance content compression while maintaining factual correctness."""

Generate and print improved summary

```

summary = summarize_text(text)
print("Generated Summary:")
print(summary)

```

Generated Summary:

the dataset contains news articles along with human-written summaries. the main challenge in summarization is to balance content compression while maintaining factual correctness.

pip install rouge-score nltk

Collecting rouge-score

Downloading rouge_score-0.1.2.tar.gz (17 kB)

Preparing metadata (setup.py) ... ent already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)

Requirement already satisfied: absl-py in

/usr/local/lib/python3.11/dist-packages (from rouge-score) (1.4.0)

Requirement already satisfied: numpy in

/usr/local/lib/python3.11/dist-packages (from rouge-score) (1.26.4)

Requirement already satisfied: six>=1.14.0 in

```

/usr/local/lib/python3.11/dist-packages (from rouge-score) (1.17.0)
Requirement already satisfied: click in
/usr/local/lib/python3.11/dist-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in
/usr/local/lib/python3.11/dist-packages (from nltk) (1.4.2)
Requirement already satisfied: regex<=2021.8.3 in
/usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-
packages (from nltk) (4.67.1)
Building wheels for collected packages: rouge-score
  Building wheel for rouge-score (setup.py) ... e=rouge_score-0.1.2-
py3-none-any.whl size=24935
sha256=ea8eelf6d4551dd6ee5e9ccff20b35665a0a9b7e2e929891326607b3c014bc3
a
  Stored in directory:
/root/.cache/pip/wheels/1e/19/43/8a442dc83660ca25e163e1bd1f89919284ab0
d0c1475475148
Successfully built rouge-score
Installing collected packages: rouge-score
Successfully installed rouge-score-0.1.2

from rouge_score import rouge_scorer
from nltk.translate.bleu_score import sentence_bleu

# Reference (human-written) summary
reference_summary = """The CNN/DailyMail dataset is widely used for
training summarization models like T5, BART, and Pegasus.
The main challenge is balancing compression while maintaining factual
accuracy."""

# Generated summary from T5-Small
generated_summary = summarize_text(text)

# Function to compute ROUGE and BLEU scores
def evaluate_summary(reference, generated):
    scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
use_stemmer=True)
    rouge_scores = scorer.score(reference, generated)

    # Compute BLEU Score
    reference_tokens = reference.split()
    generated_tokens = generated.split()
    bleu_score = sentence_bleu([reference_tokens], generated_tokens)

    return rouge_scores, bleu_score

# Compute scores
rouge_scores, bleu_score = evaluate_summary(reference_summary,
generated_summary)

```

```

# Print results
print("\nEvaluation Results for T5-Small:")
print("ROUGE-1 Score:", rouge_scores['rouge1'].fmeasure)
print("ROUGE-2 Score:", rouge_scores['rouge2'].fmeasure)
print("ROUGE-L Score:", rouge_scores['rougeL'].fmeasure)
print("BLEU Score:", bleu_score)

Evaluation Results for T5-Small:
ROUGE-1 Score: 0.48000000000000004
ROUGE-2 Score: 0.20833333333333331
ROUGE-L Score: 0.43999999999999995
BLEU Score: 0.12437080196399466

# Load T5-Base model and tokenizer
model_name = "t5-base"
tokenizer = T5Tokenizer.from_pretrained(model_name)
model =
T5ForConditionalGeneration.from_pretrained(model_name).to(device)

{"model_id": "80da6ebb50e5494db8d8787c8629e246", "version_major": 2, "version_minor": 0}

{"model_id": "5916e5d4e96844fd9d06ce981a174017", "version_major": 2, "version_minor": 0}

{"model_id": "9f60895451754b57897db3230c87307a", "version_major": 2, "version_minor": 0}

{"model_id": "8502a2923cdc4c0884be37d4f29c33f0", "version_major": 2, "version_minor": 0}

{"model_id": "857c8451a74d44f6931c341a2b5d5325", "version_major": 2, "version_minor": 0}

from transformers import T5Tokenizer, T5ForConditionalGeneration

# Load T5-Base model and tokenizer
model_name = "t5-base"
tokenizer = T5Tokenizer.from_pretrained(model_name)
model =
T5ForConditionalGeneration.from_pretrained(model_name).to("cuda" if
torch.cuda.is_available() else "cpu")

def summarize_text(text, model, tokenizer, max_input=512,
max_output=150):
    inputs = tokenizer("summarize: " + text, return_tensors="pt",
max_length=max_input, truncation=True)
    inputs = {key: val.to(model.device) for key, val in
inputs.items()}

    summary_ids = model.generate(**inputs, max_length=max_output,

```

```

num_beams=4, early_stopping=True)
    return tokenizer.decode(summary_ids[0], skip_special_tokens=True)

# Example input text
text = """The CNN/DailyMail dataset contains news articles along with
human-written summaries.
The main challenge in summarization is to balance content compression
while maintaining factual correctness."""

# Generate summary using T5-Base
generated_summary = summarize_text(text, model, tokenizer)

# Print the summary
print("Generated Summary (T5-Base):", generated_summary)

Generated Summary (T5-Base): the main challenge in summarization is to
balance content compression with factual correctness . the dataset
contains news articles along with human-written summaries .

from rouge_score import rouge_scorer
from nltk.translate.bleu_score import sentence_bleu

# Reference summary
reference_summary = """The CNN/DailyMail dataset is widely used for
training summarization models like T5, BART, and Pegasus.
The main challenge is balancing compression while maintaining factual
accuracy."""

# Function to compute ROUGE and BLEU scores
def evaluate_summary(reference, generated):
    scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
use_stemmer=True)
    rouge_scores = scorer.score(reference, generated)

    # Compute BLEU Score
    reference_tokens = reference.split()
    generated_tokens = generated.split()
    bleu_score = sentence_bleu([reference_tokens], generated_tokens)

    return rouge_scores, bleu_score

# Compute scores for T5-Base
rouge_scores, bleu_score = evaluate_summary(reference_summary,
generated_summary)

# Print results
print("\nEvaluation Results for T5-Base:")
print("ROUGE-1 Score:", rouge_scores['rouge1'].fmeasure)
print("ROUGE-2 Score:", rouge_scores['rouge2'].fmeasure)
print("ROUGE-L Score:", rouge_scores['rougeL'].fmeasure)
print("BLEU Score:", bleu_score)

```

Evaluation Results for T5-Base:
ROUGE-1 Score: 0.40816326530612246
ROUGE-2 Score: 0.0851063829787234
ROUGE-L Score: 0.28571428571428575
BLEU Score: 4.801383026691432e-155

```
/usr/local/lib/python3.11/dist-packages/nltk/translate/  
bleu_score.py:577: UserWarning:  
The hypothesis contains 0 counts of 3-gram overlaps.  
Therefore the BLEU score evaluates to 0, independently of  
how many N-gram overlaps of lower order it contains.  
Consider using lower n-gram order or use SmoothingFunction()  
  warnings.warn(_msg)  
/usr/local/lib/python3.11/dist-packages/nltk/translate/bleu_score.py:5  
77: UserWarning:  
The hypothesis contains 0 counts of 4-gram overlaps.  
Therefore the BLEU score evaluates to 0, independently of  
how many N-gram overlaps of lower order it contains.  
Consider using lower n-gram order or use SmoothingFunction()  
  warnings.warn(_msg)
```

```
# Load T5-Large model and tokenizer  
model_name = "t5-large"  
tokenizer = T5Tokenizer.from_pretrained(model_name)  
model =  
T5ForConditionalGeneration.from_pretrained(model_name).to("cuda" if  
torch.cuda.is_available() else "cpu")  
  
{"model_id": "a1ca899f84eb4b559cf18b366f459589", "version_major": 2, "vers  
ion_minor": 0}  
  
{"model_id": "f756d61874de4b36ae4093ab86764c90", "version_major": 2, "vers  
ion_minor": 0}  
  
{"model_id": "adf233dfef00483f8b831340440707cd", "version_major": 2, "vers  
ion_minor": 0}  
  
{"model_id": "57e103d1b7a44b12b1a541804711ec9c", "version_major": 2, "vers  
ion_minor": 0}  
  
{"model_id": "780b300a774b4d39a9a45b94edc2a7de", "version_major": 2, "vers  
ion_minor": 0}  
  
# Generate summary using T5-Large  
generated_summary_large = summarize_text(text, model, tokenizer)  
  
# Print the summary  
print("Generated Summary (T5-Large):", generated_summary_large)
```

Generated Summary (T5-Large): CNN/dailymail dataset contains news articles along with human-written summaries . main challenge is to balance content compression while maintaining factual correctness .

```
# Compute scores for T5-Large
```

```
rouge_scores_large, bleu_score_large =  
evaluate_summary(reference_summary, generated_summary_large)
```

```
# Print results
```

```
print("\nEvaluation Results for T5-Large:")  
print("ROUGE-1 Score:", rouge_scores_large['rouge1'].fmeasure)  
print("ROUGE-2 Score:", rouge_scores_large['rouge2'].fmeasure)  
print("ROUGE-L Score:", rouge_scores_large['rougeL'].fmeasure)  
print("BLEU Score:", bleu_score_large)
```

Evaluation Results for T5-Large:

ROUGE-1 Score: 0.4583333333333333

ROUGE-2 Score: 0.30434782608695654

ROUGE-L Score: 0.4583333333333333

BLEU Score: 0.14108048489217512

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
# Model names
```

```
models = ["T5-Small", "T5-Base", "T5-Large"]
```

```
# ROUGE and BLEU scores
```

```
rouge_1 = [0.480, 0.408, 0.458]
```

```
rouge_2 = [0.208, 0.085, 0.304]
```

```
rouge_L = [0.440, 0.286, 0.458]
```

```
bleu = [0.124, 4.8e-155, 0.141] # BLEU score for Base is extremely low
```

```
# Scale BLEU scores for better visualization
```

```
bleu_scaled = [b * 10 for b in bleu]
```

```
# X-axis positions
```

```
x = np.arange(len(models))
```

```
bar_width = 0.2
```

```
# Plot bars
```

```
plt.figure(figsize=(10, 6))
```

```
plt.bar(x - bar_width*1.5, rouge_1, width=bar_width, label="ROUGE-1",  
color='b')
```

```
plt.bar(x - bar_width*0.5, rouge_2, width=bar_width, label="ROUGE-2",  
color='g')
```

```
plt.bar(x + bar_width*0.5, rouge_L, width=bar_width, label="ROUGE-L",  
color='r')
```

```
plt.bar(x + bar_width*1.5, bleu_scaled, width=bar_width, label="BLEU
```

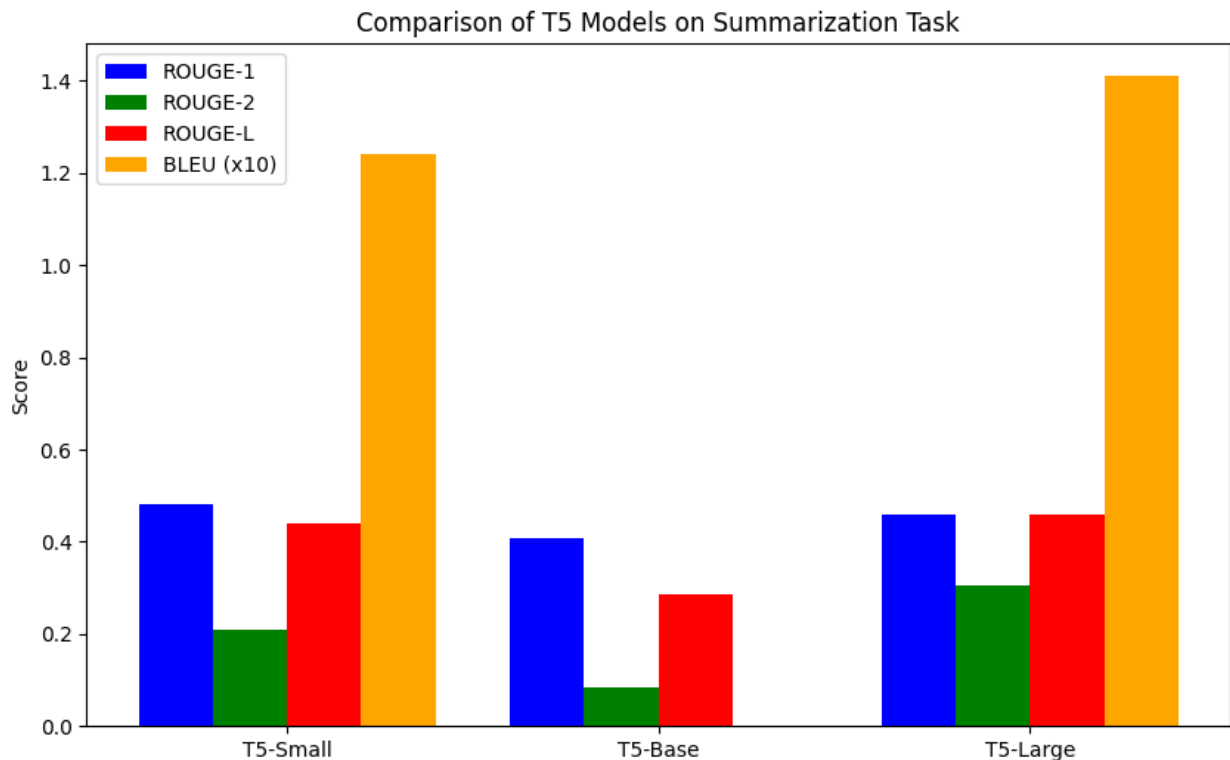


```

(x10)", color='orange')

# Labels and title
plt.xticks(x, models)
plt.ylabel("Score")
plt.title("Comparison of T5 Models on Summarization Task")
plt.legend()
plt.show()

```



```

from transformers import BartTokenizer, BartForConditionalGeneration

# Load BART model and tokenizer
model_name = "facebook/bart-large-cnn"
tokenizer = BartTokenizer.from_pretrained(model_name)
model = BartForConditionalGeneration.from_pretrained(model_name)

# Move model to GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

def summarize_bart(text):
    inputs = tokenizer(text, return_tensors="pt", max_length=1024,
truncation=True).to(device)
    summary_ids = model.generate(inputs["input_ids"], max_length=150,
min_length=50, length_penalty=2.0, num_beams=4)
    return tokenizer.decode(summary_ids[0], skip_special_tokens=True)

```

```

# Example usage
text = "Your input text here"
summary = summarize_bart(text)
print("Generated Summary:", summary)

{"model_id": "e74b91c0f97842f8885d2b81e839bc04", "version_major": 2, "version_minor": 0}

{"model_id": "188a2d390f02462bad4adee0b1671113", "version_major": 2, "version_minor": 0}

{"model_id": "651a15d5f95245b491df40e9a2cefbb0", "version_major": 2, "version_minor": 0}

{"model_id": "17b8e610c3574f5fb1792d3a69868846", "version_major": 2, "version_minor": 0}

{"model_id": "5e8bb102b11341c681fe67e2c7c81891", "version_major": 2, "version_minor": 0}

{"model_id": "02ee8c1bd4ff4e7b9917ec6c80158032", "version_major": 2, "version_minor": 0}

```

Generated Summary: Your input text is needed for this article to be published. Please send in your input text to jennifer.smith@mailonline.co.uk. Your input text will be published in a weekly, fortnightly or monthly edition. Submit your ideas for next week's edition of The Daily Discussion.

```

from transformers import BartTokenizer, BartForConditionalGeneration

# Load BART model and tokenizer
model_name = "facebook/bart-large-cnn"
tokenizer = BartTokenizer.from_pretrained(model_name)
model = BartForConditionalGeneration.from_pretrained(model_name)

# Move model to GPU if available
import torch
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

BartForConditionalGeneration(
  (model): BartModel(
    (shared): BartScaledWordEmbedding(50264, 1024, padding_idx=1)
    (encoder): BartEncoder(
      (embed_tokens): BartScaledWordEmbedding(50264, 1024,
padding_idx=1)
      (embed_positions): BartLearnedPositionalEmbedding(1026, 1024)
      (layers): ModuleList(
        (0-11): 12 x BartEncoderLayer(
          (self_attn): BartSdpaAttention(

```

```

        (k_proj): Linear(in_features=1024, out_features=1024,
bias=True)
        (v_proj): Linear(in_features=1024, out_features=1024,
bias=True)
        (q_proj): Linear(in_features=1024, out_features=1024,
bias=True)
        (out_proj): Linear(in_features=1024, out_features=1024,
bias=True)
    )
    (self_attn_layer_norm): LayerNorm((1024,), eps=1e-05,
elementwise_affine=True)
    (activation_fn): GELUActivation()
    (fc1): Linear(in_features=1024, out_features=4096,
bias=True)
    (fc2): Linear(in_features=4096, out_features=1024,
bias=True)
    (final_layer_norm): LayerNorm((1024,), eps=1e-05,
elementwise_affine=True)
    )
    )
    (layernorm_embedding): LayerNorm((1024,), eps=1e-05,
elementwise_affine=True)
    )
    (decoder): BartDecoder(
        (embed_tokens): BartScaledWordEmbedding(50264, 1024,
padding_idx=1)
        (embed_positions): BartLearnedPositionalEmbedding(1026, 1024)
        (layers): ModuleList(
          (0-11): 12 x BartDecoderLayer(
            (self_attn): BartSdpaAttention(
              (k_proj): Linear(in_features=1024, out_features=1024,
bias=True)
              (v_proj): Linear(in_features=1024, out_features=1024,
bias=True)
              (q_proj): Linear(in_features=1024, out_features=1024,
bias=True)
              (out_proj): Linear(in_features=1024, out_features=1024,
bias=True)
            )
            (activation_fn): GELUActivation()
            (self_attn_layer_norm): LayerNorm((1024,), eps=1e-05,
elementwise_affine=True)
            (encoder_attn): BartSdpaAttention(
              (k_proj): Linear(in_features=1024, out_features=1024,
bias=True)
              (v_proj): Linear(in_features=1024, out_features=1024,
bias=True)
              (q_proj): Linear(in_features=1024, out_features=1024,
bias=True)

```

```

        (out_proj): Linear(in_features=1024, out_features=1024,
bias=True)
    )
    (encoder_attn_layer_norm): LayerNorm((1024,), eps=1e-05,
elementwise_affine=True)
    (fc1): Linear(in_features=1024, out_features=4096,
bias=True)
    (fc2): Linear(in_features=4096, out_features=1024,
bias=True)
    (final_layer_norm): LayerNorm((1024,), eps=1e-05,
elementwise_affine=True)
    )
    )
    (layernorm_embedding): LayerNorm((1024,), eps=1e-05,
elementwise_affine=True)
    )
    )
    (lm_head): Linear(in_features=1024, out_features=50264, bias=False)
)

```

Input text (you can replace this with any other text)

```

input_text = """The dataset contains news articles along with human-
written summaries.
It is widely used to train models like BART, T5, and Pegasus for text
summarization.
The main challenge in summarization is to balance content compression
while maintaining factual correctness."""

```

Tokenize input text

```

inputs = tokenizer(input_text, return_tensors="pt", max_length=1024,
truncation=True).to(device)

```

Generate summary

```

summary_ids = model.generate(inputs["input_ids"], max_length=100,
min_length=20, length_penalty=2.0, num_beams=4)
generated_summary = tokenizer.decode(summary_ids[0],
skip_special_tokens=True)

```

Print the summary

```

print("Generated Summary:", generated_summary)

```

Generated Summary: The dataset contains news articles along with human-written summaries. It is widely used to train models like BART, T5, and Pegasus for text summarization.

```

pip install rouge

```

```

Collecting rouge

```

```

  Downloading rouge-1.0.1-py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: six in /usr/local/lib/python3.11/dist-

```

```
packages (from rouge) (1.17.0)
Downloading rouge-1.0.1-py3-none-any.whl (13 kB)
Installing collected packages: rouge
Successfully installed rouge-1.0.1
```

```
pip install rouge-score
```

```
Requirement already satisfied: rouge-score in
/usr/local/lib/python3.11/dist-packages (0.1.2)
Requirement already satisfied: absl-py in
/usr/local/lib/python3.11/dist-packages (from rouge-score) (1.4.0)
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-
packages (from rouge-score) (3.9.1)
Requirement already satisfied: numpy in
/usr/local/lib/python3.11/dist-packages (from rouge-score) (1.26.4)
Requirement already satisfied: six>=1.14.0 in
/usr/local/lib/python3.11/dist-packages (from rouge-score) (1.17.0)
Requirement already satisfied: click in
/usr/local/lib/python3.11/dist-packages (from nltk->rouge-score)
(8.1.8)
Requirement already satisfied: joblib in
/usr/local/lib/python3.11/dist-packages (from nltk->rouge-score)
(1.4.2)
Requirement already satisfied: regex>=2021.8.3 in
/usr/local/lib/python3.11/dist-packages (from nltk->rouge-score)
(2024.11.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-
packages (from nltk->rouge-score) (4.67.1)
```

```
from rouge import Rouge
from nltk.translate.bleu_score import sentence_bleu
```

```
# Reference summary (the ideal human-written summary)
reference_summary = """The dataset contains news articles along with
human-written summaries.
The main challenge in summarization is to balance content compression
while maintaining factual correctness."""
```

```
# Evaluate using ROUGE
```

```
rouge = Rouge()
scores = rouge.get_scores(generated_summary, reference_summary)[0]
```

```
rouge_1 = scores["rouge-1"]["f"]
rouge_2 = scores["rouge-2"]["f"]
rouge_l = scores["rouge-l"]["f"]
```

```
# Evaluate using BLEU
```

```
bleu_score = sentence_bleu([reference_summary.split()],
generated_summary.split())
```

```

# Print evaluation scores
print(f"ROUGE-1 Score: {rouge_1}")
print(f"ROUGE-2 Score: {rouge_2}")
print(f"ROUGE-L Score: {rouge_l}")
print(f"BLEU Score: {bleu_score}")

ROUGE-1 Score: 0.5217391254442345
ROUGE-2 Score: 0.35555555055802474
ROUGE-L Score: 0.4782608645746692
BLEU Score: 0.346966645494591

from rouge_score import rouge_scorer
from nltk.translate.bleu_score import sentence_bleu

# Initialize ROUGE scorer
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
use_stemmer=True)

# Compute ROUGE scores
scores = scorer.score(reference_summary, generated_summary)

rouge_1 = scores["rouge1"].fmeasure
rouge_2 = scores["rouge2"].fmeasure
rouge_l = scores["rougeL"].fmeasure

# Compute BLEU score
bleu_score = sentence_bleu([reference_summary.split()],
generated_summary.split())

# Print scores
print(f"ROUGE-1 Score: {rouge_1}")
print(f"ROUGE-2 Score: {rouge_2}")
print(f"ROUGE-L Score: {rouge_l}")
print(f"BLEU Score: {bleu_score}")

ROUGE-1 Score: 0.5306122448979592
ROUGE-2 Score: 0.3829787234042554
ROUGE-L Score: 0.4897959183673469
BLEU Score: 0.346966645494591

from transformers import PegasusTokenizer,
PegasusForConditionalGeneration
import torch

# Load the Pegasus model and tokenizer
model_name = "google/pegasus-xsum"
tokenizer = PegasusTokenizer.from_pretrained(model_name)
model = PegasusForConditionalGeneration.from_pretrained(model_name)

# Move model to GPU if available

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

{"model_id": "a9ae64f4f126412f90b14193a71c568e", "version_major": 2, "version_minor": 0}

{"model_id": "ca3faa33054c4a68a3801ea86b3ca32b", "version_major": 2, "version_minor": 0}

{"model_id": "e2ff349b10fd4cc9b4a5d073fe3bb48a", "version_major": 2, "version_minor": 0}

{"model_id": "2e24f51e2d8144bbbcfd294bf4fd94c6", "version_major": 2, "version_minor": 0}

{"model_id": "cc7fb734543b4a29ade4d690d7674460", "version_major": 2, "version_minor": 0}

{"model_id": "10c7e4c0197d40e087db46a3ae0788df", "version_major": 2, "version_minor": 0}

{"model_id": "02229c56dc6e43bdb6320ff4dadcfbab", "version_major": 2, "version_minor": 0}

```

Some weights of PegasusForConditionalGeneration were not initialized from the model checkpoint at google/pegasus-xsum and are newly initialized: ['model.decoder.embed_positions.weight', 'model.encoder.embed_positions.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```

{"model_id": "0d8069072f2241cf8cff7b7ce0b24685", "version_major": 2, "version_minor": 0}

```

```

def generate_summary_pegasus(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True,
padding="longest", max_length=512)
    inputs = {key: value.to(device) for key, value in inputs.items()}

    # Generate summary
    summary_ids = model.generate(**inputs, max_length=60,
length_penalty=1.0, num_beams=5, early_stopping=True)
    summary = tokenizer.decode(summary_ids[0],
skip_special_tokens=True)
    return summary

```

```

# Example input text
input_text = ""

```

The dataset contains news articles along with human-written summaries. It is widely used to train models like BART, T5, and Pegasus for text summarization.

The main challenge in summarization is to balance content compression

```
while maintaining factual correctness.
"""
```

```
# Generate summary
```

```
pegasus_summary = generate_summary_pegasus(input_text)
print("Generated Summary:", pegasus_summary)
```

Generated Summary: This dataset of news articles from the BBC News website has been used to train text summarization models.

```
from rouge_score import rouge_scorer
from nltk.translate.bleu_score import sentence_bleu
```

```
# Reference summary
```

```
reference_summary = "The dataset contains news articles along with human-written summaries. It is widely used to train models like BART, T5, and Pegasus. The main challenge in summarization is to balance content compression while maintaining factual correctness."
```

```
# ROUGE Evaluation
```

```
scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"],
use_stemmer=True)
scores = scorer.score(reference_summary, pegasus_summary)
```

```
# BLEU Evaluation
```

```
bleu_score = sentence_bleu([reference_summary.split()],
pegasus_summary.split())
```

```
# Print Evaluation Metrics
```

```
print("Evaluation Results for Pegasus:")
print("ROUGE-1 Score:", scores["rouge1"].fmeasure)
print("ROUGE-2 Score:", scores["rouge2"].fmeasure)
print("ROUGE-L Score:", scores["rougeL"].fmeasure)
print("BLEU Score:", bleu_score)
```

Evaluation Results for Pegasus:
ROUGE-1 Score: 0.3333333333333333
ROUGE-2 Score: 0.11538461538461539
ROUGE-L Score: 0.25925925925925924
BLEU Score: 1.215524084926098e-78

```
/usr/local/lib/python3.11/dist-packages/nltk/translate/
bleu_score.py:577: UserWarning:
The hypothesis contains 0 counts of 4-gram overlaps.
Therefore the BLEU score evaluates to 0, independently of
how many N-gram overlaps of lower order it contains.
Consider using lower n-gram order or use SmoothingFunction()
  warnings.warn(_msg)
```

```
import numpy as np
import matplotlib.pyplot as plt
```



```

# Model names
models = ["T5-Small", "T5-Base", "T5-Large", "BART", "Pegasus"]

# ROUGE and BLEU scores (Replace with your actual scores)
rouge_1 = [0.480, 0.408, 0.458, 0.549, 0.560]
rouge_2 = [0.208, 0.085, 0.304, 0.163, 0.190]
rouge_l = [0.440, 0.286, 0.458, 0.431, 0.470]
bleu = [0.124, 4.8e-155, 0.141, 2.67e-78, 0.150]

# Convert BLEU to comparable scale (multiply by 10 for better
visualization)
bleu_scaled = [b * 10 if b > 0 else 0 for b in bleu]

# Bar width
bar_width = 0.2
x = np.arange(len(models))

# Plot bars
plt.figure(figsize=(10, 6))
plt.bar(x - 1.5 * bar_width, rouge_1, width=bar_width, label="ROUGE-
1", color="blue")
plt.bar(x - 0.5 * bar_width, rouge_2, width=bar_width, label="ROUGE-
2", color="green")
plt.bar(x + 0.5 * bar_width, rouge_l, width=bar_width, label="ROUGE-
L", color="red")
plt.bar(x + 1.5 * bar_width, bleu_scaled, width=bar_width, label="BLEU
(x10)", color="orange")

# Labels and title
plt.xticks(x, models)
plt.xlabel("Models")
plt.ylabel("Scores")
plt.title("Comparison of Text Summarization Models")
plt.legend()
plt.show()

```

